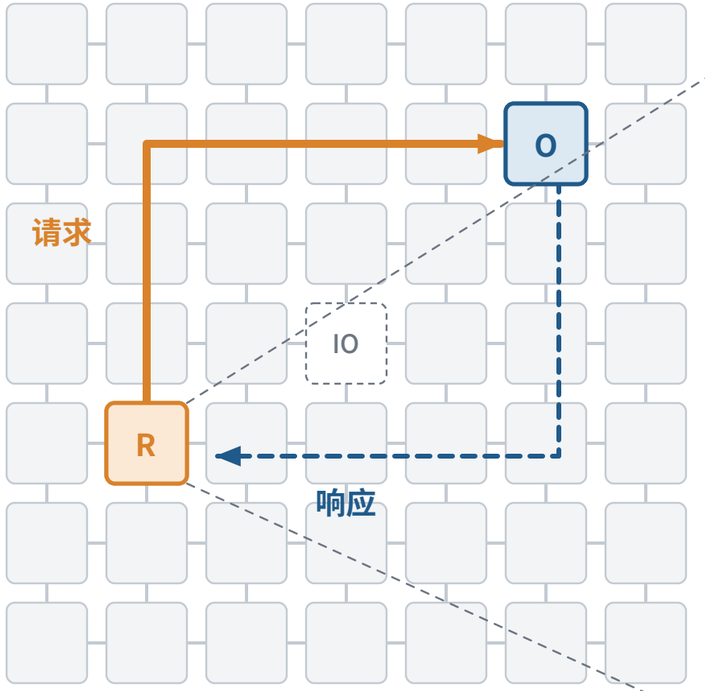
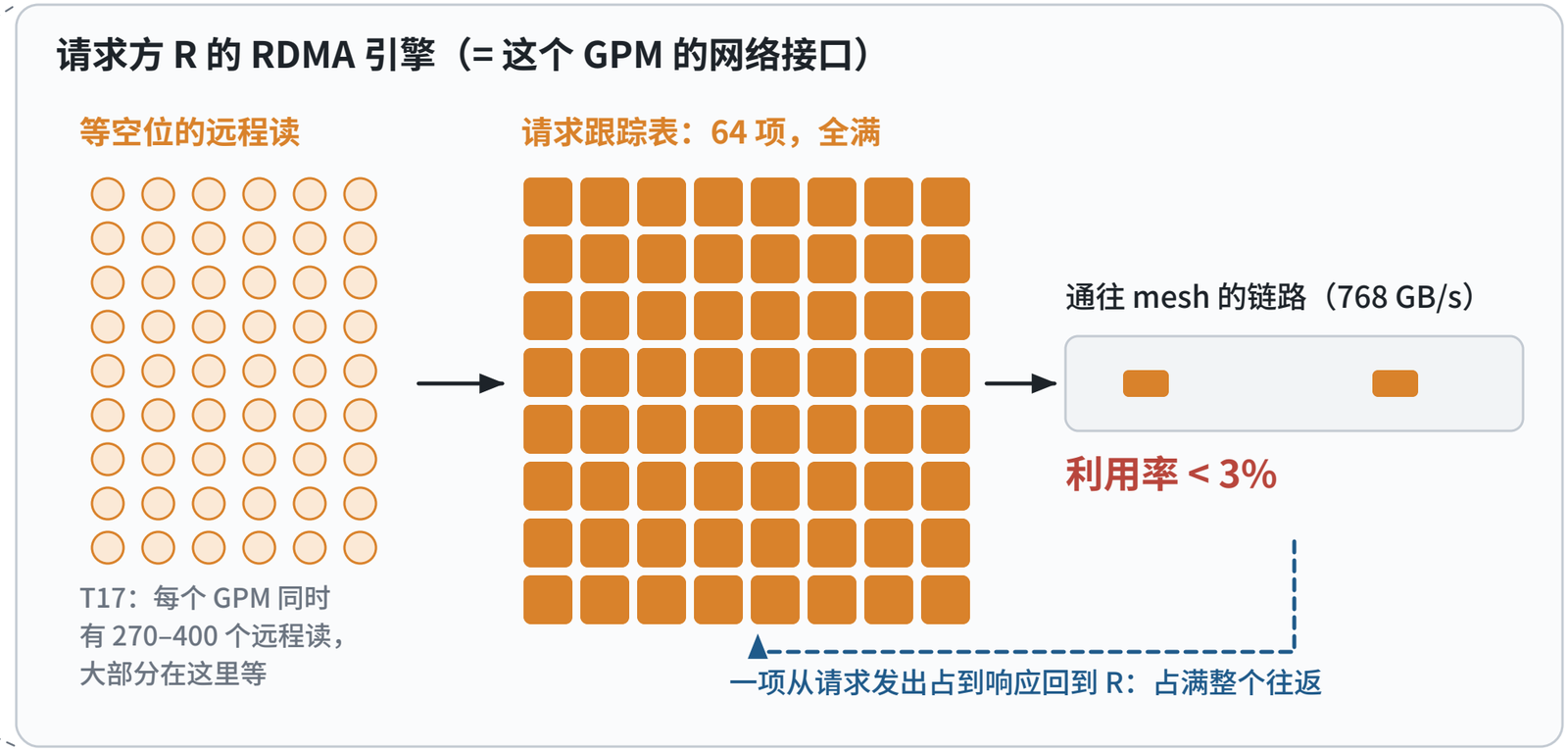
# 图 1 基线的瓶颈：请求方的 RDMA 引擎只能同时挂 64 个远程请求

RDMA 引擎就是每个 GPM 连到 mesh 的网络接口，只在两端经过；中间每一跳只过 switch，不碰 RDMA




这一次 7 跳：只有 R、O 两端经过 RDMA，
中间每跳只过 switch（约 72 ns / 跳）




### 带宽被表项卡住

跳数越多，往返越长，每项占得越久：

远程带宽 = 64 × 64 B ÷ 往返时间

1 跳 26.4 B/周期，6 跳只剩 8.0

### 时间花在排队上

按闭环模型，排队占远程延迟 80–86%

（T17 的三个 workload）

owner 端收进来的请求另有 64 项上限

### 链路是闲的

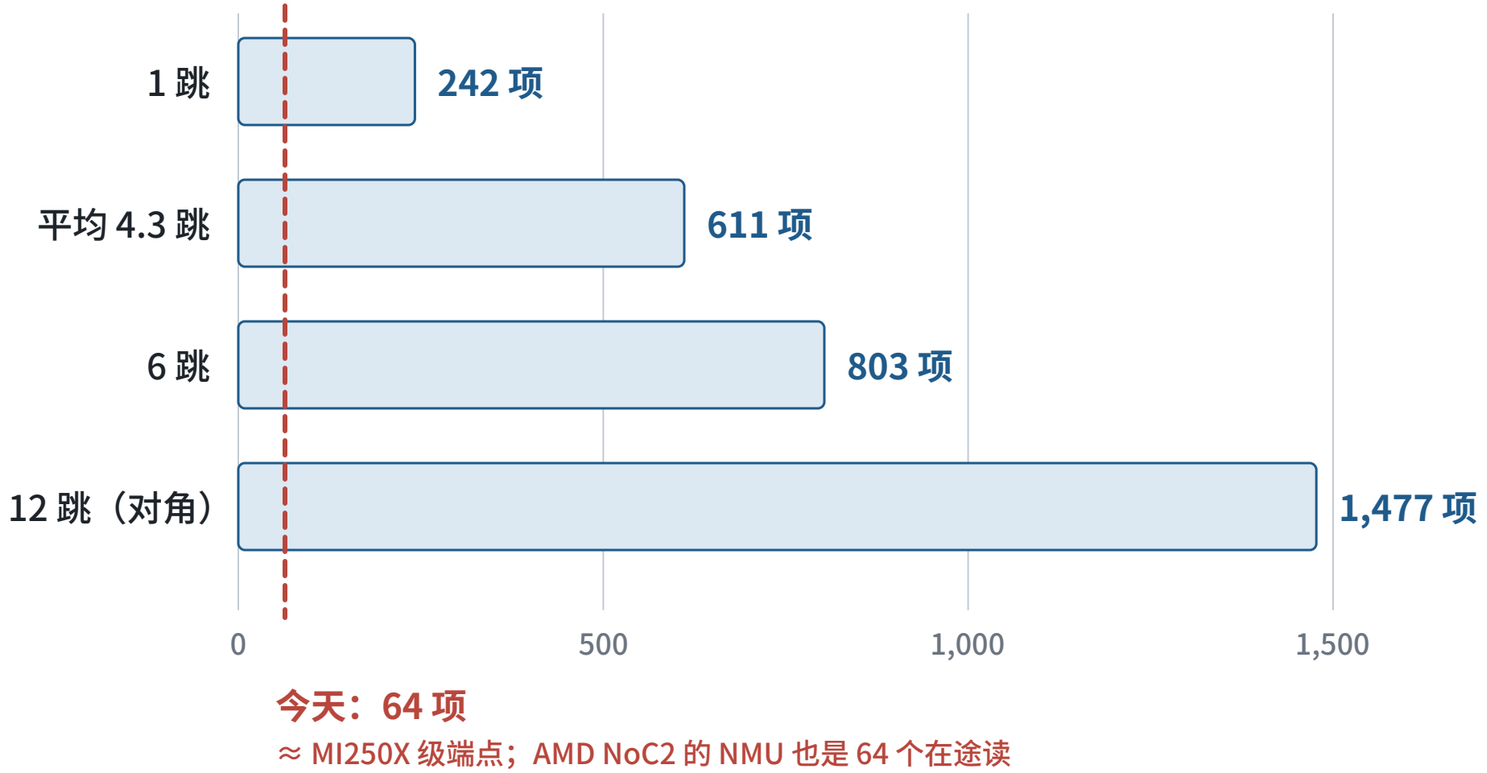
链路利用率 < 3%

受限的是两端的 RDMA，

不是链路和 switch

# 图 2 为什么 wafer 上才严重：需要的跟踪表项随距离增长

RDMA 的一项从请求发出占到响应回来；中间每跳不碰 RDMA，但跳数越多往返越长，同样的带宽就要更多项




**需要的在途数**

= 远程带宽 × 往返时间 ÷ 64 B

按每 GPM 100 B/ns 远程带宽

往返 ≈ 84 + 71.4 ns × 跳数

一个 GPM 的链路 768 GB/s，

往返 155–945 ns

**→ 填满链路要 120–730 KB 在途**

64 项 × 64 B 只有 4 KB，

**差 30–180 倍**

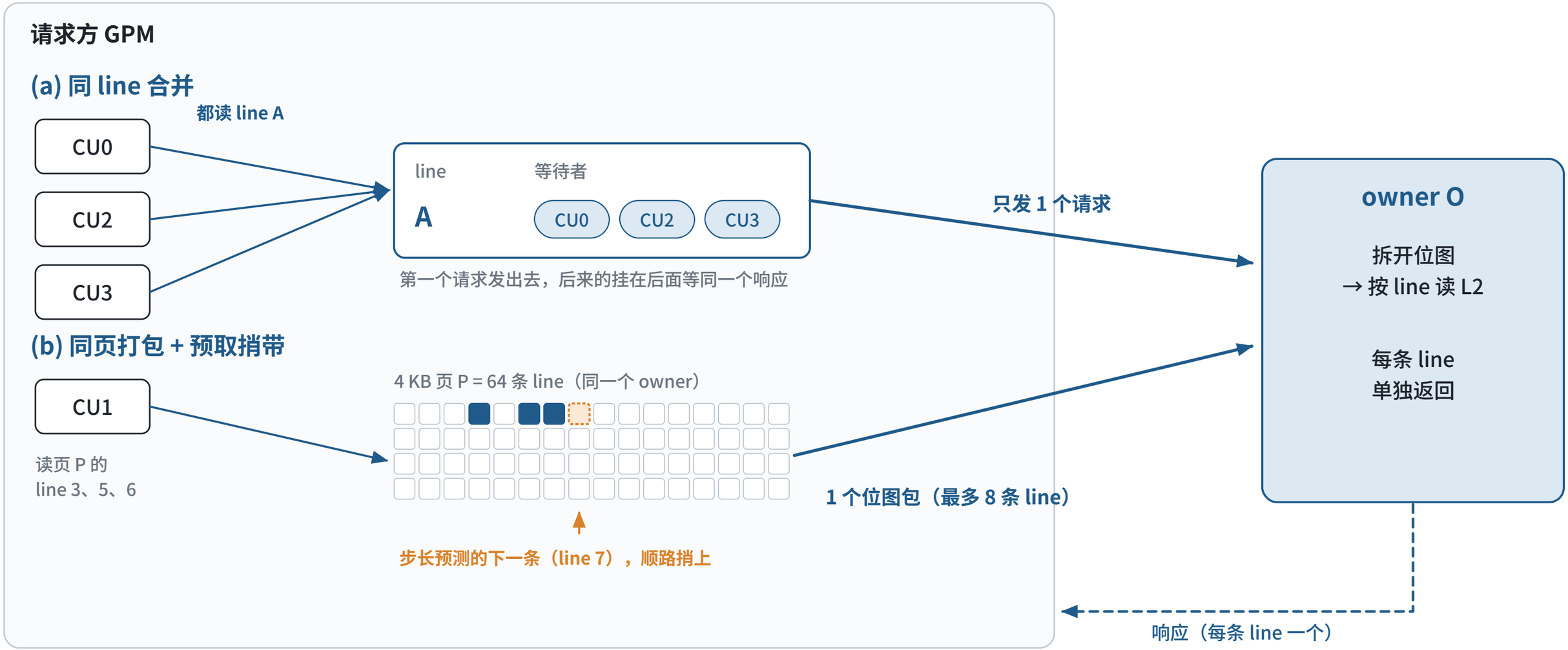
**wafer 的链路更宽、距离更远，需要的表项比多芯片 GPU 大一个数量级；全相联 CAM 做不到几千项 → M1 的出发点**

还要做：3×3 / 5×5 / 7×7 的规模实验，证明收益随 GPM 数增长（回答"为什么对 wafer 独特"）

注：MGPUSim 上游没有这个上限，64 是我们 fork 加的；论文里写成"沿用今天多芯片 GPU 的端点设计"，并扫 64 / 256 / 1024 / 4096

# 图 3 M2（HPCA 版 Remote Aggregation，已有）：在请求方合并、打包

同一个 GPM 发出的远程读，在进入 mesh 之前先去重，再把同一页的 line 打成一个包



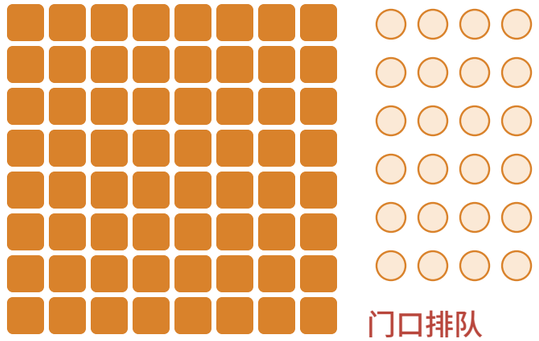

实现：每个包最多 8 条 line，最多 64 个正在收集的页；预测器只在同一 owner、同一页里预测，不多发包。

**在 Complete 里，同 line 合并交给 M1 的跟踪表（M2 自己的 filter 去重关掉），M2 保留打包和预取。**

# 图 4 M1 新 1：把请求跟踪表从 64 项做到 1024 项，用 cuckoo 索引代替 CAM

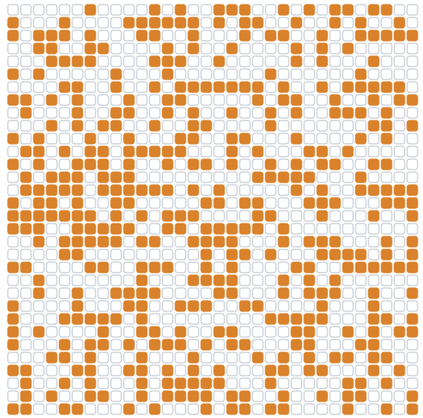
项数够了，请求不用在门口排队；查表只读两个桶，再用完整地址核对

**之前：64 项**



门口排队

**之后：1024 项**



不再排队，
链路忙起来

**一次查表（每个远程读都要查）**




指纹只做快速筛选；合并前一定核对完整地址，所以指纹撞了也不会读错数据

冒烟：W = 1024 和 4096 的结果相同，所以论文里写"几百到几千项"

**为什么不用全相联 CAM**

CAM 每次要和每一项比较，做到上千项代价很高；

cuckoo 每次只读 2 个桶（面积用 CACTI 对比，还没做）

**按距离分 credit（细节，可不讲）**

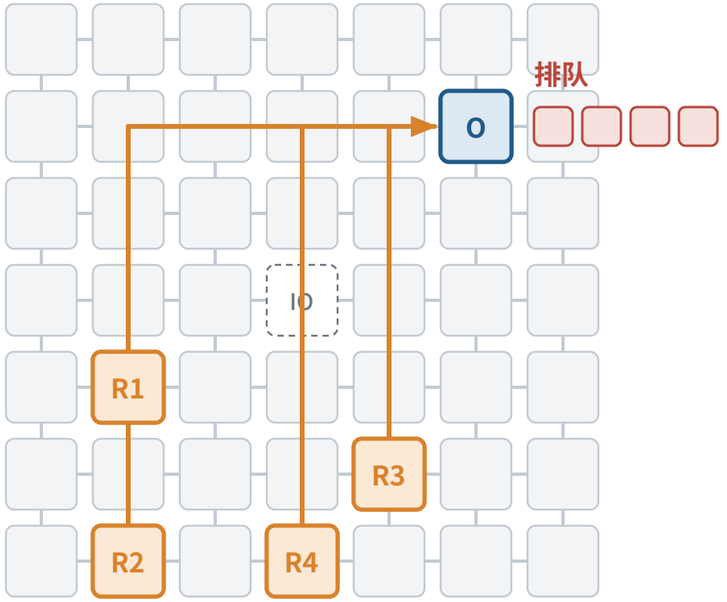
每个目的地的份额按空载往返 84 + 71.4 ns × 跳数；

空闲份额可以借；W ≥ 1024 时和共享池结果相同

# 图 5 M1 新 2：owner 端跨 GPM 合并 + 网内多播

只放大跟踪表，很多 GPM 会同时读同一条 line，瓶颈转到 owner；owner 合并后只读一次，响应沿路分叉

## (a) 只放大跟踪表

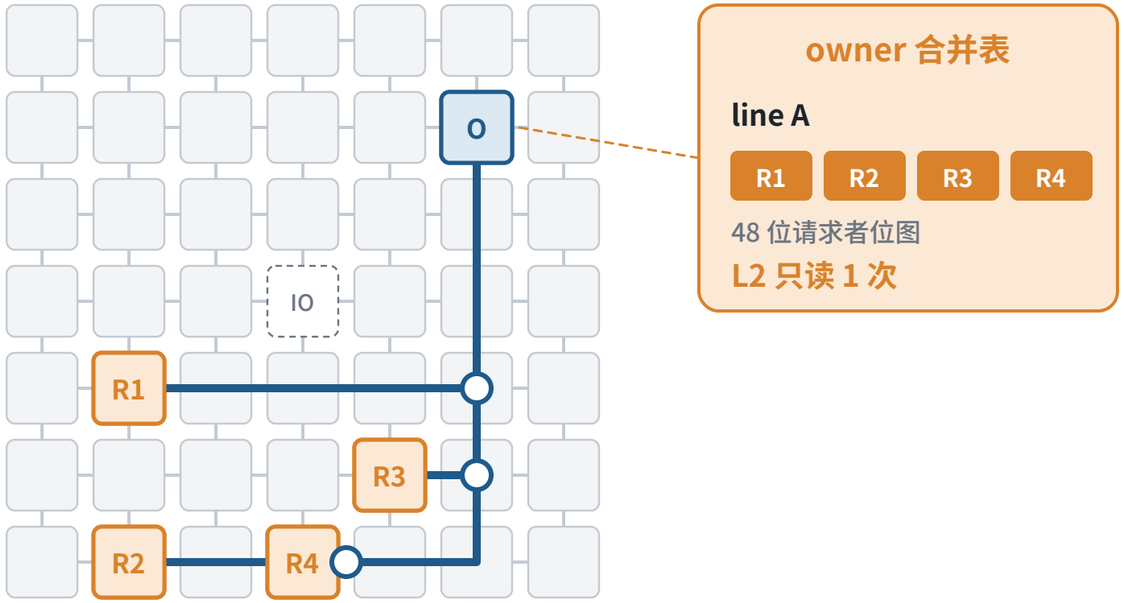



同一条 line：读 4 次 L2、回 4 份响应

incast：owner 封顶约 113 B/周期，往返停在 3–4 μs

**FW 冒烟：只放大跟踪表 2.26×**

## (b) + owner 合并 + 多播




一份响应在路由树的 switch 里复制（圆点处分叉）

链路占用：分别回 28 段 → 树 14 段

**FW 冒烟：两部分一起 8.56×（全本地上限 11.1× 的 77%）**

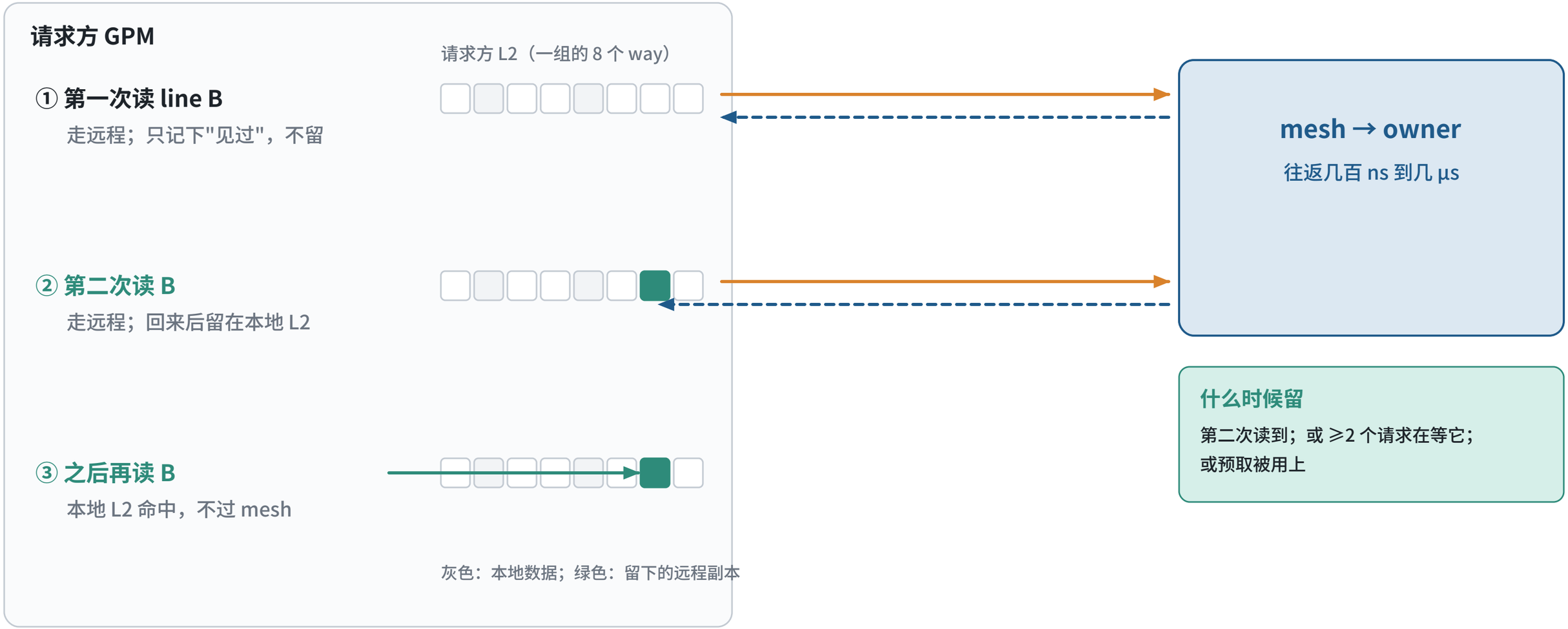
合并窗口：该 line 的 L2 读完成之前到达的同 line 请求都能并进来；表满就退回原来的单播。

为什么对 wafer 有用：同一条 line 最多 47 个 GPM 在读（4 个 GPM 的 MCM 最多 3 个）；读者之间最近只隔 1.5–2.6 跳。

**只开 owner 端几乎没用（FW 88.3 → 87.8 μs）：表项不够时请求到不了一起，两部分要一起开。**

# 图 6 M3（HPCA 版 Remote Reuse，已有，非重点）：看到复用才留在本地 L2

同一个 GPM 再次读到同一条远程 line 时，用本地 L2 里的副本，不再穿过晶圆

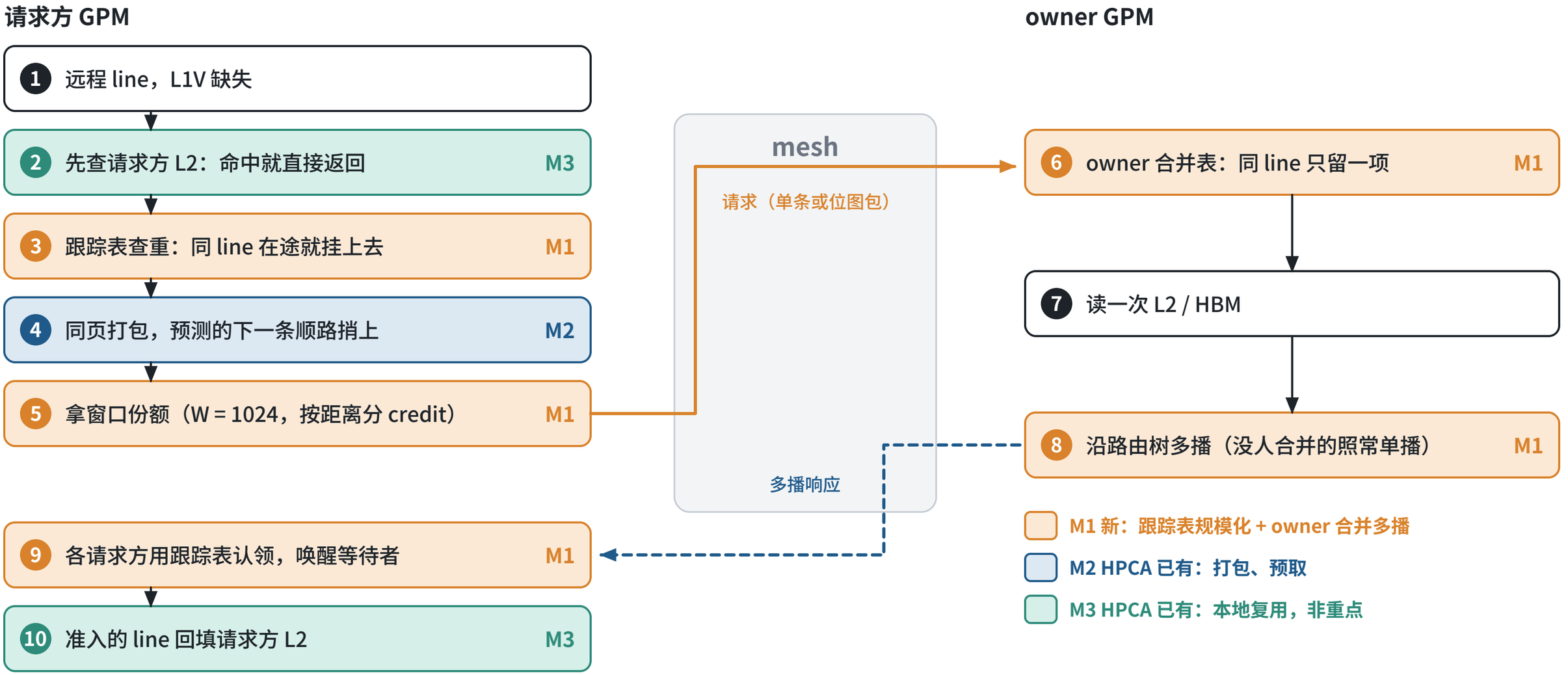



只占无效的 way 或替换别的远程副本，不挤掉本地数据；写照常发给 owner，本地副本一起更新。

还要补：kernel 边界的失效（论文写了，实现还没有）；模拟器的 L2 命中偏快（129 ns），M3 的收益可能偏高。

# 图 7 Complete = M1 + M2 + M3：一次远程读经过的每一步

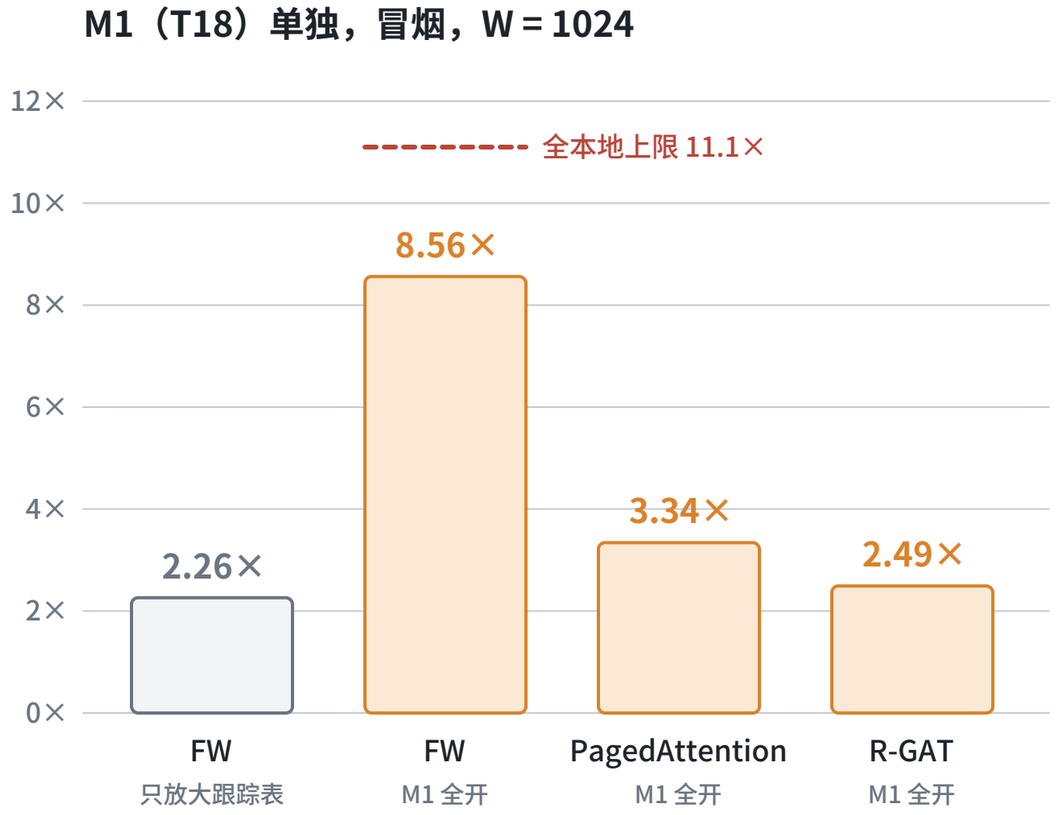
颜色表示这一步由哪个机制负责

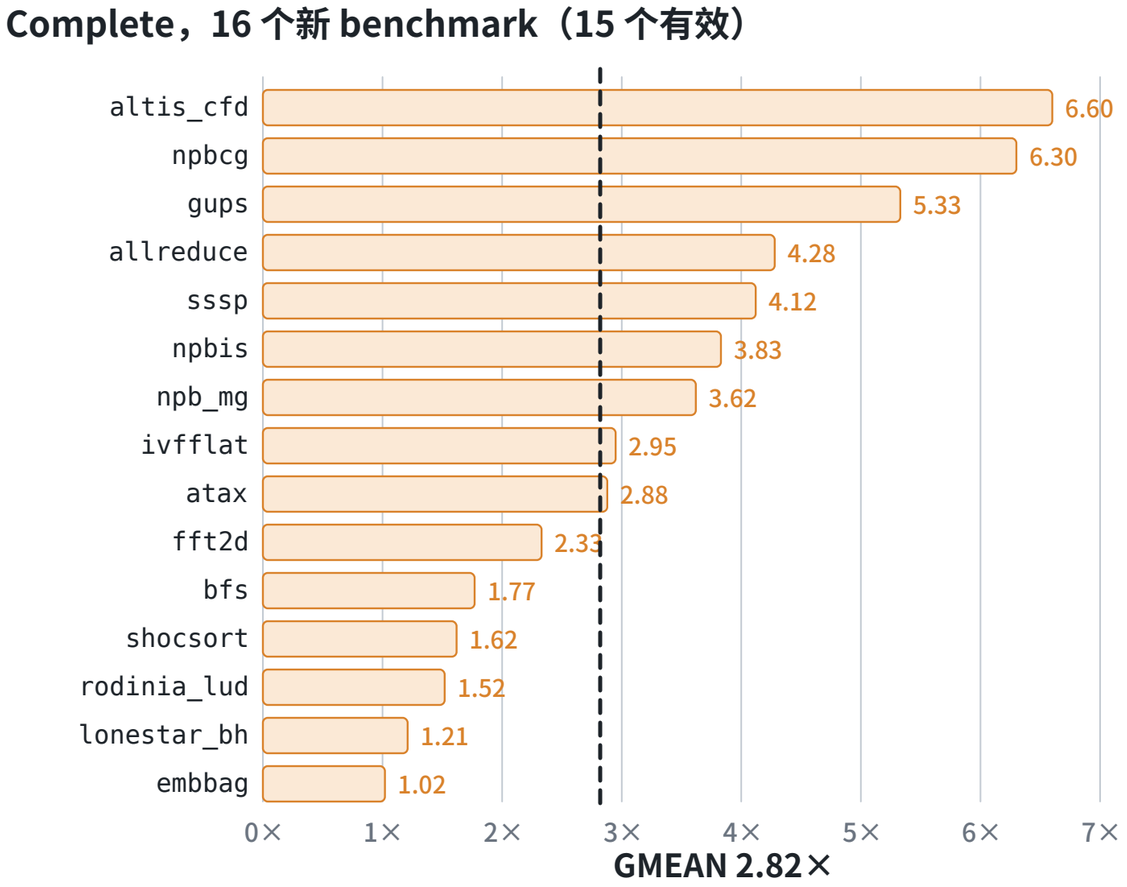



代码：isca27/m1m2 的 complete_new = T18 全开（W = 1024 + 按距离分 credit）+ 旧 m2 + 旧 m3；去重归 M1；Local Pairing 不进论文

# 图 8 初步结果（冒烟和中小规模运行，正式评估在跑）

左：M1 单独；右：Complete 在 16 个新 benchmark 上




**同样 11 个 benchmark：HPCA 版 Complete 1.88×，新 Complete 2.98×。**

都是初步数字：FW 是截断运行；16 个新 benchmark 的数据量 2–267 MB，多数小于全晶圆 L2（96 MB），加速比可能偏高；graphtc 三个配置都崩溃，没有计入。正式评估（更大的 footprint）还在跑。